

ECommerce Sales Analytics on ClickHouse

DSCI 551 Course Project — Phase 1: Proposal

Team Members: Wenxin Li, Isabelle Du, Xinyuan Huang

Date: February 20, 2026

Database Choice

ClickHouse is an open-source columnar OLAP database built by Yandex for web analytics workloads in 2009, and its engine design is described in detail by Schulze et al. (2024). MySQL keeps all columns of a row packed together in a single 16 KB page. If a query only needs price and quantity, MySQL still pulls in category, city, payment_method, and everything else from that page. ClickHouse stores each column in its own file, so that same query touches two files and ignores the rest. An e-commerce analytics workload runs aggregations over a few numeric columns across millions of rows. Columnar storage cuts a lot of unnecessary I/O in that scenario.

Internal Focus Areas

We focus on storage, indexing, and distribution because these core components determine how much data ClickHouse reads, how efficiently queries run, and how well the system scales for real e-commerce analytics workloads.

Storage and compression. ClickHouse persists data through its MergeTree engine. Inserts land as sorted, immutable “parts” on disk, and a background thread merges small parts into bigger ones over time. Each column file gets compressed separately, so a column like region with only six distinct strings shrinks far more than a column like price with near-random float values. The project measures per-column compression ratios and partition-level metadata across one million rows to show how MergeTree organizes data on disk.

Indexing and data skipping. MergeTree does not build a per-row B-tree. It stores one primary-key sample per granule (8192 rows), creating a sparse index (Abadi et al., 2008). Skip indexes (bloom filter on user_id, minmax on price) let the engine drop entire granule

groups before decompressing column data. The project compares how many granules get read when a query hits the ORDER BY prefix versus when it does not, with EXPLAIN output printed on screen.

Distribution. Two shards sit behind a Distributed table engine, coordinated by ZooKeeper. Inserts are hash-routed by user_id; reads fan out to both shards in parallel. The project inspects cross-shard aggregation pipelines, per-shard row balance, and the two-phase merge pattern that EXPLAIN PIPELINE reveals.

Application Idea

A Python command-line tool drives sixteen interactive features against a synthetic e-commerce dataset: one million orders, two years of dates, six regions, eight product categories, 100000 users. Each feature pairs a business-level query (say, monthly revenue trend or top-spending users) with EXPLAIN PLAN or EXPLAIN PIPELINE output, keeping the SQL statement and the engine's access path on screen at the same time. The features exercise columnar scans, partition pruning, skip indexes, vectorized pipelines, HyperLogLog approximation, batch inserts, and distributed aggregation.

Responsibilities

Wenxin Li handles storage analysis (compression, partitions, MergeTree write path) and data generation. Isabelle Du handles indexing analysis (sparse index, bloom filter, minmax, EXPLAIN plans) and query modules. Xinyuan Huang handles distribution analysis (cluster setup, Distributed engine, cross-shard aggregation) and the CLI.

References

Schulze, R., Schreiber, T., Yatsishin, I., Dahimene, R., & Milovidov, A. (2024).

ClickHouse—Lightning fast analytics for everyone. Proceedings of the VLDB Endowment, 17(12), 3731–3744.

Abadi, D. J., Madden, S. R., & Hachem, N. (2008). *Column-stores vs. row-stores: How different are they really?* Proceedings of the ACM SIGMOD International Conference on Management of Data, 967–980.